

CLAUDE CODE: FROM FIRST INSTALL TO LIFE AFTER PRODUCTION

Subagents, Forks And Workflows

Claude Code deep dive, episode 8

Five agents. Slower than one conversation, and several times the cost.

A subagent keeps the mess out of your window. A fork brings the background with it. A workflow moves the plan out of the conversation and into code you can rerun.

Parallelism is not free, it is not always faster, and one documented behaviour can make a single failure rerun most of your run.

Claude Code changes with every release. Every figure, default and feature name on this sheet was checked on the date below and is stated as of that date. Check the current documentation before you rely on any of it.

In scope

- What a subagent is, and the rule for when to use one
- What it does NOT inherit
- Forks, and what they buy
- Defining one, including the cost lever people miss
- Workflows, and who holds the plan
- The quality pattern moving the plan into code buys
- The caps, and the two warning thresholds
- Why five agents can finish slower than one

Out of scope

- Agent teams, which are experimental and disabled by default
- The Agent SDK surface
- Writing workflow scripts by hand
- Any claim about speedup: none is asserted here

The test is never "how many agents can I run". It is whether the work is genuinely independent.

01 The caps and thresholds, September 2026

Concurrent subagents	20 by default
Nesting depth	3 layers below the main conversation
Concurrent workflow agents	up to 16
Agents per workflow run	1,000 total
Default size guideline	fewer than 15 agents
Warning threshold	25 agents, or 1.5 million projected tokens

These are interview assumptions chosen to make the behaviour concrete. They are not published figures from any provider and no claim is made about any particular deployment.

03 The rule for when to use one

A subagent has its own context window, its own system prompt, its own tool access and its own permissions.

The documentation gives the clearest possible rule: use one when a side task would flood your main conversation with search results, logs, or file contents you will not reference again. It does that work in its own context and returns only the summary.

So the test is not "is this task big". It is: do I want the ANSWER, and not the mess it took to get there.

Use a subagent when a side task would flood your main conversation with search results, logs, or file contents you won't reference again: the subagent does that work in its own context and returns only the summary.

Claude Code docs, Subagents, checked September 2026

Each subagent runs in its own context window with a custom system prompt, specific tool access, and independent permissions.

Claude Code docs, Subagents, checked September 2026

04 What it does not inherit, which surprises everyone

A subagent does NOT inherit your conversation history. Nor your output style, your main conversation's auto memory, or any skills you invoked earlier.

What it does get: its own system prompt, the task message, your CLAUDE.md files, a git status snapshot, and any skills it was told to preload.

So when you delegate something and it comes back having missed the constraint you agreed forty minutes ago, it never saw that conversation. It is not being forgetful. It was never told.

Subagents do not inherit conversation history, output style preferences, the main conversation's auto memory, or previously invoked skills.

Claude Code docs, Subagents, checked September 2026

A non-fork subagent's initial context contains its own system prompt, the task message, CLAUDE.md files, a git status snapshot, and any preloaded skills.

Claude Code docs, Subagents, checked September 2026



05 Which is exactly what a fork is for

A fork inherits the ENTIRE conversation so far instead of starting fresh, along with the system prompt, the tools and the model. And its own tool calls still stay out of your conversation.

The documentation's rule is precise: use a fork when any other subagent would need too much background to be useful, or when you want to try several approaches in parallel from the same starting point.

That second use is the good one. Three forks from the same point, three attempts at a hard problem, and your main conversation stays exactly as it was.

A fork is a subagent that inherits the entire conversation so far instead of starting fresh; the fork's own tool calls still stay out of your conversation and only its final result comes back.

Claude Code docs, Subagents, checked September 2026

Use a fork when any other subagent would need too much background to be useful, or when you want to try several approaches in parallel from the same starting point.

Claude Code docs, Subagents, checked September 2026

06 Defining one, and the cost lever people miss

A markdown file with a small settings block, in an agents folder in your project or your home directory. Name, description, the tools it is allowed, and the model it runs on.

That last field is the lever: a subagent can run on a smaller, cheaper model than your main conversation. A file lister does not need your best model.

And keep the descriptions short, because they load at startup and cost context. Claude Code warns you when your own subagents' combined descriptions exceed fifteen thousand tokens.

Subagent descriptions take up context, and Claude Code shows a warning when the combined descriptions of your non-built-in subagents exceed 15,000 tokens.

Claude Code docs, Subagents, checked September 2026

Control costs by routing tasks to faster, cheaper models, which subagents can do via the model field.

Claude Code docs, Subagents, checked September 2026



07 Who holds the plan

With subagents, CLAUDE holds the plan and decides turn by turn what to spawn next, and every result lands in a context window.

With a workflow, the SCRIPT holds the plan, and intermediate results live in script variables. So your context holds only the final answer.

That is not a bigger version of delegation. It is moving the orchestration out of the conversation entirely, into something you can read, review and rerun.

A dynamic workflow is a JavaScript script that orchestrates many subagents at once; Claude writes the script and a runtime executes it in the background while your session stays responsive.

Claude Code docs, Dynamic workflows, checked September 2026

A workflow script holds the loop, the branching and the intermediate results itself, so Claude's context holds only the final answer.

Claude Code docs, Dynamic workflows, checked September 2026

08 And that buys a quality pattern, not just scale

Moving the plan into code lets a workflow apply a repeatable quality pattern rather than simply running more agents.

Independent agents can adversarially review each other's findings before they are reported. A plan can be drafted from several angles and the angles weighed against each other.

The bundled research workflow does exactly that: it fans out, cross checks sources against each other, and filters out claims that did not survive. Not more output. Output you have more reason to trust.

A workflow can have independent agents adversarially review each other's findings before they are reported, or draft a plan from several angles and weigh them against each other.

Claude Code docs, Dynamic workflows, checked September 2026



09 Why five agents finished slower, in three parts

First, and most importantly: if the work is not genuinely independent, you have not parallelised anything. You have added coordination to something that was already sequential.

Second, in a fan out, agents that share a prompt cache prefix are deliberately held for up to five seconds so they can read what the first one cached rather than each paying to process it. That is a saving, and it is also latency you did not expect.

Third, and this is the expensive one: if an agent fails partway through a run, relaunching reruns it AND every agent that started after it, even the ones that already completed. Start A, B, C and D; if B fails, a relaunch returns A from cache and runs B, C and D again.

When an agent failed, relaunching runs it again, and so does every agent that started after it, even ones that completed.

Claude Code docs, Dynamic workflows, checked September 2026

In a fan-out, agents that share the first agent's prompt-cache prefix start up to five seconds after it by default, so all but the first read the cached prefix.

Claude Code docs, Dynamic workflows, checked September 2026

A single run can use meaningfully more tokens than working through the same task in conversation.

Claude Code docs, Dynamic workflows, checked September 2026

Before you fan anything out

1. Is this work genuinely independent, or sequential in a parallel costume?
2. Can you check the outputs cheaply, or is that homework?
3. Would being wrong here be expensive and hard to detect?
4. Do you want the answer without the mess? That is a subagent
5. Does it need the background? That is a fork
6. Is the orchestration itself worth keeping? That is a workflow
7. Have you set a size guideline, or accepted the default?
8. What happens to your run if the third agent fails?

Everything else



Site

<https://code-with-sam-dev.github.io>



YouTube

<https://www.youtube.com/@CodewithSam-Dev>



LinkedIn

<https://www.linkedin.com/company/code-with-sam-dev>



x

<https://x.com/CodeWithSamDev>



TikTok

<https://www.tiktok.com/@codewithsamdev>



Instagram

<https://www.instagram.com/codewithsamdev>



Facebook

<https://www.facebook.com/codewithsamdev>



GitHub

<https://github.com/code-with-sam-dev>



Buy me a coffee

<https://buymeacoffee.com/codewithsam>



Buy me sadza

<https://buymesadza.com/codewithsam>

Subagents

<https://code.claude.com/docs/en/sub-agents>

If this was useful, a like and a subscribe help more than you would think. And tell me in the comments which part of your workflow you want taken apart next.

Claude and Claude Code are products of Anthropic. This is an independent, unofficial guide produced by Code with Sam. It is not affiliated with, endorsed by, or sponsored by Anthropic, and no Anthropic artwork is reproduced here. Product names appear only to identify the software being discussed. Other names are the property of their respective owners.

Technical claims verified 2026-09-13 against the documentation named beside each one. Vendors change behaviour and defaults between versions, so check the version you are actually running before you rely on any of it.